VERITAS UNIVERSITY, ABUJA · DEPARTMENT OF SOFTWARE ENGINEERING

VUA-SEN307 Database Systems — Week 6 Practical

Lab 4: Data Manipulation Language (DML): Insert, Update, Delete

Duration: 3 hours **Tools:** MySQL Server, MySQL Workbench, Git **Platforms:** Windows & macOS

Builds on: your Week 5 tables (`week05/schema.sql`)

Objectives

By the end of today you should be able to:

- populate your schema with realistic sample data using `INSERT INTO`
- insert many rows at once with a multi-row INSERT
- change records safely with `UPDATE ... WHERE`
- understand referential integrity by triggering, then fixing, a foreign-key error

Session plan

Time	Activity
0:00 – 0:10	Recap: open the Level 1 Connection and run `SHOW TABLES;`
0:10 – 0:25	**Part 1:** Insert the provided departments and courses
0:25 – 1:30	**Part 2:** Insert your own lecturers, students, enrolments and allocations
1:30 – 1:40	Break
1:40 – 2:00	**Part 3:** UPDATE a record
2:00 – 2:35	**Part 4:** The referential-integrity exercise (DELETE)
2:35 – 3:00	Final counts, write-up, GitHub and submission check

Important: Later labs depend on this data. Weeks 12 and 13 search the course **descriptions**, so insert the provided courses exactly as written. Write every statement in one script and save it as **`data.sql`** as you go.

Part 1: Departments and Courses (provided, 15 min)

Start your `data.sql` with `USE sen307_<matric_number>;`, then add and run this block. Each course finds its department by name, so you don't need to know the ID numbers.

```
INSERT INTO Department (dept_name) VALUES
('Software Engineering'),
('Computer Science'),
('Artificial Intelligence'),
('Data Science'),
('Cybersecurity');

INSERT INTO Course (course_code, course_title, credit_units, description, dept_id) VALUES
('SEN307', 'Database Systems', 3,
 'Relational database design, SQL queries, normalization, transactions and indexing using
MySQL.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Software Engineering')),
('SEN301', 'Software Architecture', 3,
 'Architectural styles, design patterns, microservices and how to structure large software
systems.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Software Engineering')),
('COS301', 'Operating Systems', 3,
```

```
 'Processes, threads, CPU scheduling, memory management and file systems.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Computer Science')),
('COS305', 'Computer Networks', 3,
 'TCP/IP, routing, switching, LAN and WAN design and network protocols.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Computer Science')),
('AIN301', 'Introduction to Artificial Intelligence', 3,
 'Intelligent agents, search algorithms, knowledge representation and reasoning.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Artificial Intelligence')),
('AIN305', 'Machine Learning', 3,
 'Supervised and unsupervised learning, neural networks, deep learning and model
evaluation.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Artificial Intelligence')),
('AIN307', 'Natural Language Processing', 3,
 'Text processing, word embeddings, language models, sentiment analysis and chatbots.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Artificial Intelligence')),
('DTS301', 'Data Visualization and Analytics', 2,
 'Exploratory data analysis, charts, dashboards and telling stories with data.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Data Science')),
('DTS305', 'Big Data Technologies', 3,
 'Distributed storage and processing of very large datasets with Hadoop and Spark.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Data Science')),
('CYB301', 'Network Security', 3,
 'Firewalls, cryptography, intrusion detection and protecting networks from attacks.',
 (SELECT dept_id FROM Department WHERE dept_name = 'Cybersecurity'));

SELECT d.dept_name, COUNT(c.course_id) AS courses
FROM Department d LEFT JOIN Course c ON d.dept_id = c.dept_id
GROUP BY d.dept_name;
```

The final query should list 5 departments with 10 courses between them. Next, note the ID numbers you will need:

```
SELECT dept_id, dept_name FROM Department;
SELECT course_id, course_code FROM Course;
```

Part 2: Your Own Data (65 min)

Write your own INSERT statements using realistic Nigerian names. Use **multi-row INSERTs**, which put many rows in one statement. Check your `dept_id` values against the output of the query above.

Step 1: Lecturers (10 in total)

```
INSERT INTO Lecturer (staff_no, full_name, email, dept_id) VALUES
('VUA/ST/101', 'Dr Amaka Okafor', 'a.okafor@vua.edu.ng', 1),
('VUA/ST/102', 'Dr Musa Bello',   'm.bello@vua.edu.ng',  1);
-- ...add 8 more lecturers (10 in total), spread across all 5 departments
```

Step 2: Students (20 in total)

Include your own name and matric number as one of the students.

```
INSERT INTO Student (matric_no, full_name, gender, date_of_birth, dept_id) VALUES
('20231234', 'Ada Obi',   'F', '2005-03-14', 1),
('20231235', 'John Musa', 'M', '2004-11-02', 2);
-- ...add 18 more students (20 in total)
```

Step 3: Enrolments (40 in total) and course allocations

First list the IDs, then insert using those numbers:

```
SELECT student_id, matric_no, full_name FROM Student;
SELECT lecturer_id, full_name FROM Lecturer;

INSERT INTO Enrolment (student_id, course_id) VALUES
(1, 1), (1, 2), (2, 1), (2, 3);
-- ...add more rows until you have 40 enrolments in total

INSERT INTO CourseAllocation (course_id, lecturer_id) VALUES
(1, 1), (1, 2), (2, 2);
-- ...give every course at least one lecturer; course 1 has two
```

Your data must include (later labs need these):

- at least **one course with more than 5 students** (Week 8, HAVING)
- at least **two students with no enrolments** (Weeks 7 and 8, LEFT JOIN and subqueries)
- at least **one course taught by two lecturers** in CourseAllocation
- a mix of genders and dates of birth between 2003 and 2007 (Week 7, filtering)
- no student enrolled twice in the same course; the UNIQUE constraint will reject it

Part 3: UPDATE a Record (20 min)

Move a student to another department, and check the change before and after:

```
SELECT matric_no, full_name, dept_id FROM Student WHERE matric_no = '20231234';

UPDATE Student SET dept_id = 2 WHERE matric_no = '20231234';

SELECT matric_no, full_name, dept_id FROM Student WHERE matric_no = '20231234';
```

Error 1175 (safe update mode)? MySQL Workbench blocks some UPDATE and DELETE statements by default. Wrap the statement like this, then switch the protection back on:

```
SET SQL_SAFE_UPDATES = 0;
-- ...run your UPDATE or DELETE...
SET SQL_SAFE_UPDATES = 1;
```

Part 4: Referential Integrity (35 min)

Step 1: Create a department with a student in it

```
INSERT INTO Department (dept_name) VALUES ('Library and Information Science');

INSERT INTO Student (matric_no, full_name, dept_id)
VALUES ('20239999', 'Transfer Student',
        (SELECT dept_id FROM Department WHERE dept_name = 'Library and Information
Science'));
```

Step 2: Try to delete the department

```
DELETE FROM Department WHERE dept_name = 'Library and Information Science';
```

You should see **Error 1451: Cannot delete or update a parent row: a foreign key constraint fails**. Take a screenshot of it. MySQL refuses because a student still points to this department.

Step 3: Fix it the correct way

```
-- Step 1: reassign the dependent student to another department
UPDATE Student
SET dept_id = (SELECT dept_id FROM Department
               WHERE dept_name = 'Software Engineering')
WHERE matric_no = '20239999';

-- Step 2: now the delete succeeds
DELETE FROM Department WHERE dept_name = 'Library and Information Science';

SELECT * FROM Department;
```

You are back to 5 departments, and the student has been moved rather than lost.

Final Check and Write-up (25 min)

```
SELECT 'Departments' AS tbl, COUNT(*) AS total FROM Department
UNION ALL SELECT 'Lecturers',   COUNT(*) FROM Lecturer
UNION ALL SELECT 'Students',    COUNT(*) FROM Student
UNION ALL SELECT 'Courses',     COUNT(*) FROM Course
UNION ALL SELECT 'Enrolments',  COUNT(*) FROM Enrolment
UNION ALL SELECT 'Allocations', COUNT(*) FROM CourseAllocation;
```

You should have at least 5 departments, 10 lecturers, 20 students, 10 courses, 40 enrolments and 10 allocations. Take a screenshot of this result.

Write-up (`lab4_notes.md`, a short paragraph): What error did you get in Part 4? Why does MySQL refuse? How did you fix it, and what other fix could you have used?

Push to GitHub (same on both)

1. Create this week's folder in your repository:

   ```
   cd sen307-labs
   mkdir week06
   ```

2. Copy `data.sql`, `lab4_notes.md` and your screenshots into `week06/`. On Windows, use File Explorer; on macOS, use Finder.
3. Commit and push:

   ```
   git add .
   git commit -m "Week 6: DML sample data and referential integrity"
   git push
   ```

Submission Checklist (before you leave)

- ☐ `week06/data.sql` with every INSERT, UPDATE and DELETE you ran
- ☐ `week06/lab4_notes.md` explaining the referential-integrity error and how you fixed it
- ☐ Screenshots of Error 1451 and the final row counts
- ☐ All files pushed to GitHub

Next week (Week 7): you will query this data with SELECT and JOINs. Don't delete your data.